

UNIVERSIDAD AUTÓNOMA DE ZACATECAS

INGENIERÍA DE SOFTWARE

DEPLOYMENT

PROYECTO OBSERVABILIDAD

DANIEL ALONSO MARTÍNEZ CASTILLO

01/06/2026



PREGUNTAS DE REFLEXIÓN TÉCNICA

1. ¿Cuál es la diferencia entre monitoreo, observabilidad y alertamiento?

El monitoreo consiste en recolectar y visualizar métricas predefinidas de un sistema para conocer su estado en un momento dado. La observabilidad es la capacidad de entender el comportamiento interno de un sistema a partir de sus salidas externas (métricas, logs y trazas), permitiendo diagnosticar fallos que no fueron anticipados, y el alertamiento es el mecanismo que notifica automáticamente a los operadores cuando una métrica supera un umbral crítico. En esta práctica, Prometheus provee el monitoreo, Grafana la observabilidad visual, y Alertmanager el alertamiento por correo.

2. ¿Por qué Prometheus trabaja bajo un modelo de scraping?

Prometheus usa un modelo de pull (scraping) en lugar de push porque simplifica el control: es el propio Prometheus quien decide cuándo y con qué frecuencia consulta cada target. Esto facilita la detección de servicios caídos (si un target no responde al scrape, se marca como DOWN automáticamente), evita sobrecargas por múltiples agentes enviando datos simultáneamente, y centraliza la configuración de intervalos en un solo lugar. Además, el modelo pull es más seguro en redes internas porque los exporters no necesitan conocer la dirección del servidor de monitoreo.

3. ¿Qué es un exporter y por qué se necesita?

Un exporter es un proceso intermediario que traduce las métricas propias de un servicio (que no habla el formato de Prometheus) al formato de texto de Prometheus (/metrics). Se necesita porque la mayoría de aplicaciones y sistemas operativos no exponen métricas en ese formato de forma nativa. Por ejemplo, MariaDB expone sus estadísticas internamente mediante variables de sistema, y mysqld_exporter las consulta y las convierte. Lo mismo ocurre con node_exporter para el sistema operativo Linux, cadvisor para Docker, y jmx_prometheus_javaagent para la JVM de Tomcat.

4. ¿Qué diferencia hay entre monitorear el host y monitorear contenedores?

Monitorear el host con node_exporter da una visión global de los recursos físicos o de la VM: CPU total, RAM disponible, disco, red y carga del sistema. Es el límite máximo de recursos que todo lo demás comparte. Monitorear contenedores con cAdvisor da una visión granular por proceso aislado: cuánta CPU y memoria consume cada contenedor individualmente, sus reinicios y su estado. Ambas perspectivas son necesarias: el host detecta saturación general, y los contenedores identifican cuál servicio específico es el responsable del consumo.

5. ¿Qué métricas son más importantes para detectar saturación del servidor?

Las métricas más críticas para detectar saturación son: el uso de CPU (node_cpu_seconds_total), que indica procesamiento excesivo; la memoria disponible (node_memory_MemAvailable_bytes), ya que al agotarse el sistema empieza a usar

swap y degrada el rendimiento; el espacio en disco (`node_filesystem_avail_bytes`), cuyo llenado puede detener servicios completos; la carga promedio del sistema (`node_load1`, `node_load5`, `node_load15`), que refleja la demanda acumulada sobre los CPUs; y el tráfico de red (`node_network_receive_bytes_total`), que puede indicar congestión o ataques.

6. ¿Qué métricas son relevantes para una aplicación Django?

Con `django-prometheus` las métricas más relevantes son: el número de requests HTTP (`django_http_requests_total_by_method_total`), que mide la carga de la aplicación; la latencia de requests (`django_http_requests_latency_seconds`), que detecta degradación de rendimiento; los códigos de respuesta HTTP (`django_http_responses_total_by_status_total`), especialmente errores 4xx y 5xx; las consultas a base de datos (`django_db_execute_total`), que revelan cuellos de botella en queries; y la disponibilidad del endpoint `/metrics` mismo, que sirve como indicador de que la aplicación está viva.

7. ¿Qué métricas son relevantes para Tomcat/JVM?

Las métricas clave expuestas por `jmx_prometheus_javaagent` son: la memoria heap y non-heap de la JVM (`jvm_memory_bytes_used`), esencial para detectar fugas de memoria; el Garbage Collection (`jvm_gc_collection_seconds_total`), cuya frecuencia excesiva indica presión de memoria; los threads activos (`jvm_threads_current`), que revelan concurrencia y posibles deadlocks; las solicitudes procesadas por Tomcat y su tiempo de respuesta, que miden throughput; los errores HTTP del conector de Tomcat; y el estado general del proceso JVM mediante `up`.

8. ¿Qué riesgos existen al exponer Prometheus, Grafana o exporters públicamente?

Exponer estos servicios públicamente sin autenticación representa riesgos graves: cualquier persona podría leer métricas sensibles del sistema (uso de recursos, nombres de servicios, versiones de software), que facilitan el reconocimiento previo a un ataque. Prometheus y los exporters no tienen autenticación nativa, por lo que un atacante podría hacer scraping masivo de datos o intentar manipular configuraciones. Grafana, si no se asegura, permite acceso a dashboards y potencialmente a la ejecución de queries. Por eso el PDF recomienda exponer solo Grafana con dominio y proxy inverso, y mantener Prometheus, Alertmanager y exporters en red interna únicamente.

9. ¿Por qué no deben guardarse contraseñas SMTP en el repositorio?

Porque un repositorio Git mantiene historial permanente: aunque se borre el archivo, la contraseña queda registrada en commits anteriores y puede recuperarse con `git log` o `git show`. Si el repositorio es público (o se hace público en el futuro), cualquier persona puede extraer las credenciales. Incluso en repos privados, múltiples personas suelen tener acceso. La práctica correcta es usar variables de entorno cargadas desde un archivo `.env` excluido del repo con `.gitignore`, o un gestor de secretos como AWS Secrets Manager o HashiCorp Vault en producción.

10. ¿Qué mejoras implementarían si esta solución se llevara a producción?

Las mejoras prioritarias serían:

- 1) Autenticación y TLS en Prometheus y Alertmanager mediante proxy inverso con NGINX o Traefik;
- 2) Gestión de secretos con AWS Secrets Manager o variables de entorno inyectadas por CI/CD en lugar de archivos .env.
- 3) Persistencia de datos de Prometheus con volúmenes de largo plazo y retención configurada
- 4) Alta disponibilidad con Prometheus en modo HA o usando Thanos para federación
- 5) Grafana con provisioning automático de dashboards y datasources como código.
- 6) Alertas más granulares con distintos canales según severidad (PagerDuty para critical, Slack para warning)
- 7) Healthchecks y readiness probes en todos los contenedores.